WEEK 2 : Spring Core and Maven, Spring Data JPA and hibernate - Mandatory Hands on

Spring core and Maven:

Exercise 1: Configuring a Basic Spring Application

Step 1: pom.xml

```
<dependencies>
   <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>5.3.30</version>
   </dependency>
</dependencies>
```

Step 2: BookRepository.java

```
package com.library.repository;

public class BookRepository {

   public void displayBook() {
      System.out.println("Book Repository Working");
   }
}
```

Step 3: BookService.java

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

   private BookRepository bookRepository;

   public void setBookRepository(BookRepository
bookRepository) {
      this.bookRepository = bookRepository;
   }

   public void showBook() {
      bookRepository.displayBook();
   }
}
```

Step 4: applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans
xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
      xsi:schemaLocation="
      http://www.springframework.org/schema/beans
      https://www.springframework.org/schema/beans/spring-
```

```
beans.xsd">

    <bean id="bookRepository"
        class="com.library.repository.BookRepository"/>

    <bean id="bookService"
        class="com.library.service.BookService"/>

</beans>
```

Step 5: LibraryManagementApplication.java

```
package com.library;

import org.springframework.context.ApplicationContext;
import
org.springframework.context.support.ClassPathXmlApplicatio
nContext;
import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =
                new
ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service = (BookService)
```

```
context.getBean("bookService");

        System.out.println("Spring Configuration Loaded
Successfully");
    }
}
```

Exercise 2: Implementing Dependency Injection

BookService.java

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository
bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void showBook() {
        bookRepository.displayBook();
    }
}
```

---

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans
xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
      xsi:schemaLocation="
      http://www.springframework.org/schema/beans
      https://www.springframework.org/schema/beans/spring-
beans.xsd">

   <bean id="bookRepository"
         class="com.library.repository.BookRepository"/>

   <bean id="bookService"
         class="com.library.service.BookService">

       <property name="bookRepository"
               ref="bookRepository"/>

   </bean>

</beans>
```

LibraryManagementApplication.java

```
package com.library;
```

```
import org.springframework.context.ApplicationContext;
import
org.springframework.context.support.ClassPathXmlApplicatio
nContext;
import com.library.service.BookService;

public class LibraryManagementApplication {

   public static void main(String[] args) {

      ApplicationContext context =
            new
ClassPathXmlApplicationContext("applicationContext.xml");

      BookService service =
            context.getBean("bookService", BookService.class);

      service.showBook();
   }
}
```

Output

Book Repository Working

Exercise 4: Creating and Configuring a Maven Project

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
        xsi:schemaLocation="
        http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.library</groupId>
    <artifactId>LibraryManagement</artifactId>
    <version>1.0</version>

    <dependencies>

        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>5.3.30</version>
        </dependency>

        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-aop</artifactId>
            <version>5.3.30</version>
        </dependency>

        <dependency>
            <groupId>org.springframework</groupId>
```

```
        <artifactId>spring-webmvc</artifactId>
        <version>5.3.30</version>
      </dependency>

    </dependencies>

    <build>
      <plugins>

        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.8.1</version>

          <configuration>
            <source>1.8</source>
            <target>1.8</target>
          </configuration>

        </plugin>

      </plugins>
    </build>

</project>
```

Spring data JPA and Hibernate:

Hands-on 1: Spring Data JPA – Quick Example

1. Create Spring Boot Project

- Project Name: orm-learn
- Group: com.cognizant
- Artifact: orm-learn
- Dependencies:
    - Spring Boot DevTools
    - Spring Data JPA
    - MySQL Driver

---

2. pom.xml

```
<dependencies>

   <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
   </dependency>

   <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-devtools</artifactId>
   </dependency>

   <dependency>
```

```
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <scope>runtime</scope>
    </dependency>

</dependencies>
```

---

3. application.properties

```
logging.level.org.springframework=info
logging.level.com.cognizant=debug

logging.level.org.hibernate.SQL=trace
logging.level.org.hibernate.type.descriptor.sql=trace

spring.datasource.driver-class-
name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.
MySQL5Dialect
```

---

4. Create Database

```
create database ormlearn;
```

```
use ormlearn;

create table country(
co_code varchar(2) primary key,
co_name varchar(50)
);

insert into country values('IN','India');
insert into country values('US','United States');
```

---

5. Country.java

```
package com.cognizant.ormlearn.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="country")
public class Country {

    @Id
    @Column(name="co_code")
    private String code;

    @Column(name="co_name")
    private String name;
```

```
public Country() {

}

public Country(String code, String name) {
   this.code = code;
   this.name = name;
}

public String getCode() {
   return code;
}

public void setCode(String code) {
   this.code = code;
}

public String getName() {
   return name;
}

public void setName(String name) {
   this.name = name;
}

@Override
public String toString() {
   return "Country [code=" + code + ", name=" + name +
```

```
"]";
   }

}
```

---

6. CountryRepository.java

```
package com.cognizant.ormlearn.repository;

import
org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.cognizant.ormlearn.model.Country;

@Repository
public interface CountryRepository extends
JpaRepository<Country,String>{

}
```

---

7. CountryService.java

```
package com.cognizant.ormlearn.service;

import java.util.List;

import
org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.stereotype.Service;
import
org.springframework.transaction.annotation.Transactional;

import com.cognizant.ormlearn.model.Country;
import
com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public List<Country> getAllCountries(){

        return countryRepository.findAll();

    }

}
```

---

8. OrmLearnApplication.java

```
package com.cognizant.ormlearn;

import java.util.List;
```

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplicati
on;
import org.springframework.context.ApplicationContext;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootApplication
public class OrmLearnApplication {

    private static final Logger LOGGER =
            LoggerFactory.getLogger(OrmLearnApplication.class);

    private static CountryService countryService;

    public static void main(String[] args) {

        ApplicationContext context =

SpringApplication.run(OrmLearnApplication.class,args);

        countryService=context.getBean(CountryService.class);

        testGetAllCountries();
```

```
        LOGGER.info("Inside Main");

    }

    private static void testGetAllCountries(){

        LOGGER.info("Start");

        List<Country>
countries=countryService.getAllCountries();

        LOGGER.debug("Countries={}",countries);

        LOGGER.info("End");

    }

}
```

Output

```
Start

Countries=[
Country [code=IN, name=India],
Country [code=US, name=United States]
]

End
```

Inside Main

Hands-on 5: Implement Services for Managing Country

Update CountryService.java

```
@Service
public class CountryService {

    @Autowired
    CountryRepository countryRepository;

    @Transactional
    public List<Country> getAllCountries(){

        return countryRepository.findAll();

    }

}
```

Hands-on 6: Find Country by Code

CountryNotFoundException.java

```
package com.cognizant.ormlearn.exception;

public class CountryNotFoundException extends Exception{
```

```
    public CountryNotFoundException(String message){
        super(message);
    }

}
```

CountryService.java

```
@Transactional
public Country findCountryByCode(String code)
        throws CountryNotFoundException{

    Optional<Country> result=
            countryRepository.findById(code);

    if(result.isEmpty()){
        throw new CountryNotFoundException("Country Not
Found");
    }

    return result.get();

}
```

Test

```
Country country=countryService.findCountryByCode("IN");

System.out.println(country);
```

Output

```
Country [code=IN, name=India]
```

Hands-on 7: Add Country

CountryService.java

```
@Transactional
public void addCountry(Country country){

   countryRepository.save(country);

}
```

Test

```
Country country=new Country("JP","Japan");

countryService.addCountry(country);
```

Output

Country Added Successfully

Hands-on 8: Update Country

CountryService.java

```
@Transactional
public void updateCountry(String code,String name)
      throws CountryNotFoundException{

   Country country=findCountryByCode(code);

   country.setName(name);
```

```
    countryRepository.save(country);

}
```

Test

```
countryService.updateCountry("JP","Japan Updated");
```

Output

Country Updated Successfully

Hands-on 9: Delete Country

CountryService.java

```
@Transactional
public void deleteCountry(String code){

    countryRepository.deleteById(code);

}
```

Test

```
countryService.deleteCountry("JP");
```

Output:

Country Deleted Successfully